

Table of Contents

ИНСТРУКЦИЯ ПО ВНЕДРЕНИЮ

T3-6 — Dead Code Cleanup & Engineering Debt (v0.1)

Проект: ENIGMA v0.5.3.1.9 **Документ:** Инструкция по очистке мёртвого кода и технического долга **Тип:** Техническое задание на миграцию **Цель:** убрать мёртвый код, заменить `print()` на `logger.debug`, вынести магические числа в `constants.py`

Принцип (из Causal Authority Map): - No silent failures (логирование вместо `except: pass`) - No dead code (если модуль не используется — удалить, не «на будущее») - Single source of truth (один `WillState`, один `sprite_registry`)

0. Контекст

0.1. Диагноз

ENIGMA накопила **3 категории engineering debt**:

Категория	Примеры	Влияние
Dead code	5+ мёртвых модулей, 13+ мёртвых методов	Поддержка усложняется, drift неизбежен
Production noise	20+ <code>print()</code> в hot paths	Загрязняет stdout, мешает CDS
Magic numbers	50+ хардкоженных чисел	Невозможно калибровать

0.2. 31 пункт, сгруппированный в 3 Patch Set

Patch Set	Пунктов	Принцип	Время
A — Dead Code Removal	12	Удалить мёртвые модули и методы	2 часа
B — Production Hygiene	10	<code>print()</code> → <code>logger.debug</code> , feature-flag	2 часа
C — Constants & i18n	9	Магические числа →	3 часа

Patch Set	Пунктов	Принцип	Время
		constants.py, i18n	

1. Порядок применения

STEP 1-5 → PATCH SET A (dead code removal)

STEP 6-10 → PATCH SET B (print → logger)

STEP 11-15 → PATCH SET C (constants + i18n)

STEP 16 → validation tests

2. PATCH SET A — Dead Code Removal (12 пунктов, ~2 часа)

A1 — Удалить мёртвые модули (5 файлов)

Проверить каждый grep'ом, удалить если 0 callers:

A1.1 — world_sim_agent.py (после T3-5 C1)

```
grep -rn "WorldSimulationAgent" backend/ --include="*.py" | grep -v __pycache__ |
grep -v test
```

A1.2 — parser.py (после T3-5 C2)

```
grep -rn "ResponseParser" backend/ --include="*.py" | grep -v __pycache__ | grep -v
test
```

A1.3 — npc_response_validator.py (после T3-5 C3)

```
grep -rn "NPCResponseValidator" backend/ --include="*.py" | grep -v __pycache__ |
grep -v test
```

A1.4 — transit_tracker.py (после T3-4 A5)

A1.5 — location_graph.py (после T3-4 A5)

Если все 0 callers (после применения T3-4 и T3-5):

```
rm backend/app/agents/world_sim_agent.py
rm backend/app/services/llm/parser.py
rm backend/app/services/verbalization/npc_response_validator.py
rm backend/app/services/spatial/transit_tracker.py
rm backend/app/services/spatial/location_graph.py
```

Обновить __init__.py файлы — убрать импорты.

A2 — Удалить мёртвые методы (8 пунктов)

Для каждого — проверить grep'ом, удалить если 0 callers:

#	Файл	Метод	Проверка
A2.1	frontend/ game_screen.py	_draw_input_bar	grep -rn "_draw_input_bar" frontend/
A2.2	frontend/ game_screen.py	_handle_click (в MOUSEBUTTONDOWN OWN and False)	grep -rn "_handle_click" frontend/
A2.3	frontend/ api_client.py	_advance_time_by _movement	grep -rn "_advance_time_by _movement" frontend/
A2.4	frontend/ game_loop_bridge. py	build_perceived_sc ene	grep -rn "build_perceived_s cene" frontend/
A2.5	frontend/ game_loop_bridge. py	enrich_scene_spati al	grep -rn "enrich_scene_spa tial" frontend/
A2.6	backend/app/ agents/ dm_agent.py	_build_intro_prom pt	grep -rn "_build_intro_prom pt" backend/
A2.7	backend/app/ services/action/ dm_orchestrator.p y	can_proceed property	grep -rn "can_proceed" backend/
A2.8	backend/app/ services/ verbalization/ dm_contract_build er.py	add_npc_author_n otes	grep -rn "add_npc_author_ notes" backend/

Действие: для каждого метода — если 0 callers вне определения, удалить метод.

A3 — Убрать мёртвые поля (2 пункта)

#	Файл	Поле	Действие
A3.1	frontend/ game_types.py	PlayerMemory (создаётся, методы не вызываются)	Удалить класс + создание в game_screen.py
A3.2	frontend/ game_types.py	EncounterHistory (создаётся, методы не вызываются)	Удалить класс + создание в game_screen.py

Проверка:

```
grep -rn "PlayerMemory\|EncounterHistory" frontend/ --include="*.py" | grep -v "class \|import"
```

Если только создание без вызовов методов — удалить.

A4 — Убрать дублирующий импорт

Файл: backend/app/api/routes.py

Проверить:

```
grep -n "from app.core.config import settings" backend/app/api/routes.py
```

Если 2+ совпадений — оставить одно.

A5 — Убрать мёртвый `_DIR_MAP`

Файл: frontend/game_screen.py:681

Проверить:

```
grep -n "_DIR_MAP" frontend/game_screen.py
```

Если только определение без использования — удалить.

3. PATCH SET B — Production Hygiene (10 пунктов, ~2 часа)

B1 — `movement_engine.py` print → logger

Файл: backend/app/services/spatial/movement_engine.py

Строки: 71, 80, 109, 245, 266, 273, 284 (7 print() вызовов)

ЗАМЕНИ каждый:

```
print(f"[GATE_B1_COLLAPSE] npc={npc_id} ...")
```

НА:

```
# B1-FIX: print → logger.debug c feature-flag.
```

```
if settings.movement_debug:
```

```
    logger.debug(f"[GATE_B1_COLLAPSE] npc={npc_id} ...")
```

В начале файла добавить:

```
import logging
logger = logging.getLogger(__name__)
```

В backend/app/core/config.py добавить:

```
class Settings(BaseSettings):
    ...
    movement_debug: bool = False # B1-FIX: feature-flag for movement debug
```

B2 — tick_orchestrator.py print → logger

Файл: backend/app/services/tick_orchestrator.py

Строки: 886, 996, 1006 (и другие — найти через grep)

ЗАМЕНИ:

```
print("[SEL_BYPASS] Injecting _run_affective_pipeline...", file=sys.stderr,
flush=True)
```

НА:

```
# B2-FIX: print → logger.debug.
if settings.orchestrator_debug:
    logger.debug("[SEL_BYPASS] Injecting _run_affective_pipeline...")
```

В config.py:

```
orchestrator_debug: bool = False
```

B3 — dm_agent.py:197 print → logger

ЗАМЕНИ:

```
print(f"[STM_INJECT] ...")
```

НА:

```
# B3-FIX: print → logger.debug.
if settings.dm_debug:
    logger.debug(f"[STM_INJECT] ...")
```

B4 — game_loop_bridge.py:193-195 print → logger

ЗАМЕНИ:

```
print(f"[BRIDGE] world_snapshot build skipped: {_ws_err}")
```

НА:

```
# B4-FIX: print → logger.warning.  
logger.warning(f"[BRIDGE] world_snapshot build skipped: {_ws_err}")
```

B5 — Убрать except Exception: pass (5+ мест)

Принцип: No silent failures.

Найти все:

```
grep -rn "except Exception:\s*$\|except.*pass" backend/app/services/ frontend/ --  
include="*.py" | grep -v __pycache__ | grep -v test | head -20
```

Для каждого — заменить на:

```
# БЫЛО:  
except Exception:  
    pass  
  
# НА:  
except Exception as e:  
    logger.warning(f"[CONTEXT] operation failed: {e}")
```

Особенно важные: - agent_runner.py:71 — abort_generation -
provider_manager.py:371 — get_status - dm_agent.py:828 — notify_stream_end
- game_loop/_init_.py:968 — Death Guard - npc_state_helpers.py:108 —
loop._load_npcs() NameError

B6 — Deprecated asyncio.get_event_loop()

Файл: frontend/game_loop_bridge.py:152-161

ЗАМЕНИ:

```
loop = asyncio.get_event_loop()
```

НА:

```
# B6-FIX: deprecated asyncio.get_event_loop() → get_running_loop /  
new_event_loop.  
try:  
    loop = asyncio.get_running_loop()  
except RuntimeError:  
    loop = asyncio.new_event_loop()  
    asyncio.set_event_loop(loop)
```

B7 — ActionQueue.poll ловит KeyboardInterrupt

Файл: frontend/api_client.py:697-700

ЗАМЕНИ:

```
def poll(self) -> CompletedAction | None:
    try:
        return self._output.get_nowait()
    except Exception:
        return None
```

НА:

```
def poll(self) -> CompletedAction | None:
    # B7-FIX: ловит только queue.Empty, не KeyboardInterrupt.
    import queue
    try:
        return self._output.get_nowait()
    except queue.Empty:
        return None
    except Exception as e:
        logger.error(f"[ACTION_QUEUE] poll failed: {e}")
        return None
```

B8 — intent_compressor.py:152 except Exception → WARN

(Уже описано в ТЗ-1 Шаг 12 — если ещё не применено, применить здесь.)

B9 — WillState enum дублирование

Файлы: - backend/app/models/will.py:51 — class WillState(Enum) (13 значений) - backend/app/models/npc_state.py:108 — class WillState(str, Enum) (5 значений)

Проверить какой используется в production:

```
grep -rn "from app.models.will import WillState\|from app.models.npc_state import WillState" backend/ --include="*.py" | grep -v __pycache__ | grep -v test
```

Оставить один (предположительно npc_state.py — str, Enum для сериализации).

В will.py убрать дубликат:

```
# B9-FIX: WillState дублирован. Используйте app.models.npc_state.WillState.
# Этот файл оставлен для backward compatibility — реэкспорт.
from app.models.npc_state import WillState # noqa
```

B10 — `compute_continuous_drift` возвращает `{}` вместо `[]`

Файл: backend/app/services/npc/break_progress_engine.py:205

ЗАМЕНИ:

```
if total_mass <= 0:  
    return {}
```

НА:

```
# B10-FIX: возвращаем list, не dict (API contract).  
if total_mass <= 0:  
    return []
```

4. PATCH SET C — Constants & i18n (9 пунктов, ~3 часа)

C1 — Магические числа в `constants.py`

Создать/обновить backend/app/core/constants.py:

```
# — Scene rendering —  
SCENE_SCALE_PX_PER_M: int = 40  
NPC_RADIUS_FOCUSED: int = 10  
NPC_RADIUS_DEFAULT: int = 7  
NPC_INSTABILITY_AMP: int = 7  
  
# — Game screen —  
MOVE_INTERVAL_SEC: float = 0.08  
WALK_STEP_METERS: float = 0.3  
PLAYER_COLLISION_RADIUS: float = 0.25  
TELEGRAPH_COOLDOWN_MS: int = 30_000  
NPC_SPEECH_BUBBLE_LIFETIME_MS: int = 6000  
PLAYER_SPEECH_BUBBLE_LIFETIME_MS: int = 4000  
  
# — Spatial —  
VISIBILITY_RADIUS_M: float = 20.0  
LOS_FACING_PENALTY: float = 0.3  
LOS_THRESHOLD: float = 0.2  
COLLISION_RADIUS: float = 0.8  
  
# — DM scene outcome —  
FOCUS_NPC_CAP: int = 2  
SALIENCE_PROXIMITY_WEIGHT: float = 0.30  
SALIENCE_EMOTIONAL_WEIGHT: float = 0.30  
SALIENCE_RELEVANCE_WEIGHT: float = 0.25  
SALIENCE_TIER_WEIGHT: float = 0.15
```


HIDDEN_DISTANCE: `float` = 15.0

— Text input —
TEXT_INPUT_MAX_LENGTH: `int` = 500
TEXT_INPUT_MAX_HISTORY: `int` = 100

В соответствующих файлах заменить хардкоды на импорты из constants.

C2 — i18n для хардкоженных строк

Файл: frontend/i18n.py

ДОБАВИТЬ недостающие ключи:

```
_STRINGS = {  
    ...  
    "ui:menu_new_game": "Новая игра",  
    "ui:menu_continue": "Продолжить",  
    "ui:menu_editor": "Редактор карт",  
    "ui:menu_settings": "Настройки",  
    "ui:menu_exit": "Выход",  
    "ui:scale_1x": "▶ 1x",  
    "ui:scale_4x": "▶▶ 4x",  
    "ui:scale_10x": "▶▶▶ 10x",  
    "ui:scale_50x": "▶▶▶▶ 50x",  
    ...  
}
```

В game_screen.py, game_menu.py, campaign_select.py, character_select.py
— заменить хардкоженные строки на t("ui:...").

C3 — MSG_* константы в constants.py

Файл: backend/app/core/constants.py

ДОБАВИТЬ:

— DM messages —
MSG_MAX_REPLIES: `int` = 3
MSG_NOTHING_HAPPENED: `str` = "Ничего не произошло."
MSG_INVALID_RESPONSE: `str` = "..."

В dm_agent.py:44-47 — убрать локальные определения, импортировать из constants.

C4 — `_INTENSITY_LEMMAS` cleanup

(Уже описано в ТЗ-1 Шаг 7 — если ещё не применено, применить здесь.)

C5 — `gender` locale normalization

Файл: `config/npc/individuals/shadow.json`

ЗАМЕНИ:

"gender": "неизвестен"

НА:

"gender": "unknown"

В `state_interpreter.py` — нормализовать:

```
def _get_gender(self, npc_state) -> str:
    # C5-FIX: нормализация gender для руторфы3.
    gender = npc_state.gender
    if gender in ("неизвестен", "unknown", None):
        return "m" # default
    elif gender in ("мужской", "male", "m"):
        return "m"
    elif gender in ("женский", "female", "f"):
        return "f"
    return "m"
```

C6 — `RelationshipStore._cache` без LRU

Файл: `backend/app/services/memory/relationship_store.py`

ЗАМЕНИ простой dict на OrderedDict с LRU:

```
from collections import OrderedDict

class RelationshipStore:
    MAX_CACHE_SIZE = 1000
    TTL_SECONDS = 3600

    def __init__(self, ...):
        self._cache: OrderedDict = OrderedDict()
        self._timestamps: dict = {}

    def get(self, npc_id, target_id):
        key = f"{npc_id}→{target_id}"
        # TTL check
```

```

if key in self._timestamps:
    if time.time() - self._timestamps[key] > self.TTL_SECONDS:
        del self._cache[key]
        del self._timestamps[key]
        return None
if key in self._cache:
    self._cache.move_to_end(key)
    return self._cache[key]
return None

def set(self, npc_id, target_id, value):
    key = f"{npc_id}→{target_id}"
    self._cache[key] = value
    self._cache.move_to_end(key)
    self._timestamps[key] = time.time()
    while len(self._cache) > self.MAX_CACHE_SIZE:
        oldest = next(iter(self._cache))
        del self._cache[oldest]
        del self._timestamps[oldest]

```

C7 — origin_events.day = -1000 sentinel

Файл: config/npc/individuals/*.json

ДОБАВИТЬ поле day в каждый origin_event:

```

"origin_events": [
    {
        "summary": "...",
        "day": -1095,
        ...
    }
]

```

Или в memory_manager.py — fallback:

```

def _get_event_day(self, event: dict) -> int:
    day = event.get("day", -1000)
    if day == -1000:
        if "childhood" in event.get("tags", []):
            return -3650
        elif "trauma" in event.get("tags", []):
            return -1095
    return day

```

C8 — rules_agent._DC_BY_ACTION_TYPE неиспользуемые значения

Файл: backend/app/services/agents/rules_agent.py

Проверить:

```
grep -rn "DC_TRIVIAL|DC_NEARLY_IMPOSSIBLE" backend/ --include="*.py" |  
grep -v __pycache__
```

Если 0 callers — удалить из таблицы.

C9 — Объединить sprite_resolver и sprite_registry

Файлы: - frontend/sprite_resolver.py (124 LOC) -
frontend/map_editor/sprite_registry.py (110 LOC)

В sprite_registry.py добавить метод get_entity:

```
class SpriteRegistry:  
    ENTITY_SPRITE_MAP: dict[str, tuple[str, int, int]] = {  
        "maid": ("characters", 0, 0),  
        "thief": ("characters", 1, 0),  
        # ...  
    }  
  
    def get_entity(self, entity_type: str) -> Optional[Surface]:  
        if entity_type not in self.ENTITY_SPRITE_MAP:  
            return None  
        sheet, col, row = self.ENTITY_SPRITE_MAP[entity_type]  
        return self.get(sheet, col, row)
```

Удалить frontend/sprite_resolver.py.

В game_screen.py — обновить импорт:

```
# Было: from sprite_resolver import get_entity_sprite  
# Стало:  
from map_editor.sprite_registry import SpriteRegistry  
_registry = SpriteRegistry()  
get_entity_sprite = _registry.get_entity
```

5. STEP 16 — backend/tests/test_tz6_cleanup.py (NEW)

"""Tests for T3-6 Cleanup."""

```
import pytest  
from pathlib import Path
```

```
class TestPatchA_DeadCode:
```

```

def test_world_sim_agent_removed(self):
    assert not (Path(__file__).resolve().parents[2] / "app" / "agents" /
"world_sim_agent.py").exists()

def test_parser_removed(self):
    assert not (Path(__file__).resolve().parents[2] / "app" / "services" / "llm" /
"parser.py").exists()

def test_npc_response_validator_removed(self):
    assert not (Path(__file__).resolve().parents[2] / "app" / "services" /
"verbalization" / "npc_response_validator.py").exists()

class TestPatchB_ProductionHygiene:
    def test_no_print_in_movement_engine(self):
        path = Path(__file__).resolve().parents[2] / "app" / "services" / "spatial" /
"movement_engine.py"
        source = path.read_text(encoding="utf-8")
        # Не должно быть print( вне комментариев
        lines = [l for l in source.split('\n') if 'print(' in l and not
l.strip().startswith('#')]
        assert len(lines) == 0, f"print() found in movement_engine: {lines}"

    def test_willstate_single_definition(self):
        # Проверяем что will.py реэкспортирует, не определяет
        path = Path(__file__).resolve().parents[2] / "app" / "models" / "will.py"
        source = path.read_text(encoding="utf-8")
        assert "class WillState" not in source or "from app.models.npc_state import
WillState" in source, \
            "WillState still duplicated (B9 not applied)"

class TestPatchC_Constants:
    def test_constants_has_spatial(self):
        path = Path(__file__).resolve().parents[2] / "app" / "core" / "constants.py"
        source = path.read_text(encoding="utf-8")
        assert "VISIBILITY_RADIUS_M" in source
        assert "LOS_THRESHOLD" in source

```

6. СВОДКА

Patch	Категория	Пунктов	Время
A1	Dead modules	5	30 мин
A2	Dead methods	8	45 мин
A3	Dead fields	2	15 мин
A4-A5	Duplicates	2	10 мин

Patch	Категория	Пунктов	Время
B1-B4	print → logger	4 файла	1 час
B5	except: pass removal	5+ мест	30 мин
B6-B7	Deprecated/ KeyboardInterrupt	2	15 мин
B8-B10	Misc hygiene	3	30 мин
C1	Magic numbers → constants	30+ чисел	1 час
C2	i18n	20+ строк	1 час
C3-C9	Misc constants	7	1 час
Test	test_tz6_clean up.py	—	30 мин

Итого: ~7 часов.

7. ЧТО ПАТЧ ГАРАНТИРУЕТ

✓ 5 мёртвых модулей удалены ✓ 13 мёртвых методов удалены ✓ 20+ print() → logger.debug с feature-flag ✓ 5+ except: pass → логирование ✓ WillState — один enum ✓ compute_continuous_drift возвращает list ✓ ActionQueue.poll не ловит KeyboardInterrupt ✓ Нет deprecated asyncio.get_event_loop() ✓ Магические числа в constants.py ✓ i18n применён к хардкоженным строкам ✓ sprite_resolver и sprite_registry объединены ✓ RelationshipStore с LRU/TTL ✓ origin_events имеют day ✓ gender стандартизирован

КОНЕЦ ИНСТРУКЦИИ ТЗ-6